Tier 2 results — dataset scale

Items 30–38 of TASKS.md, executed 2026-10-12. AWS was ruled out by the author, so items 32 and 34 could not be done; everything else was completed locally.

Status

#	Item	Status
30	One-command collection harness	**Done** — `scripts/collection/collect.py`, used for every collection below
31	IMDSv2 token bug	**Done**, plus identity assertion and DOP-matched probing
32	30+ machines	**BLOCKED** — AWS vCPU quota is 1.0
33	21 → ~230 queries	**Done — 234 queries**, all validating
34	SF1/SF10/SF100	**Not done** — traded for query diversity on 31 GB of disk
35	Concurrency levels	**Done** — 1, 2, 4, 8 clients
36	Postgres version matrix	**Done** — 14.19, 15.17, 17.11, 18.6
37	Store raw full plan JSON	**Done** — with plan-drift detection
38	Publish the dataset	**Packaged**, not published — needs the author's account

The workload is now 234 queries

workload	queries	validating	database
TPC-H SF1 (q15 restored)	22	**22/22**	`tpch`, 1705 MB
TPC-DS SF1	99	**99/99**	`tpcds`, 2358 MB
Join Order Benchmark (IMDB)	113	**113/113**	`imdbload`, 8700 MB

TPC-DS was generated locally with `dsdgen` from gregrahn/tpcds-kit (which needed `-std=gnu89` to build at all — it is pre-C99 K&R C that modern clang rejects). No PostgreSQL dialect ships with the kit, so one was added as a copy of `netezza.tpl`, whose `limit %d` form Postgres accepts. 92 of 99 queries planned unmodified; the 7 failures were two well-known incompatibilities, both fixed without changing semantics:

- `subquery in FROM must have an alias` (ds2, ds14, ds23, ds49) — the SQL standard does not require a derived-table alias, PostgreSQL does. ds14's was introduced by a comma in the FROM list rather than by `from (`, which needed a second pattern.
- `column "lochierarchy" does not exist` (ds36, ds70, ds86) — these reference a SELECT-list alias inside a CASE expression in ORDER BY, which PostgreSQL does not expose.

The fixer is idempotent (it skips files already carrying its header), after a first version double-aliased on re-run and broke three queries.

JOB required no fixes — 113/113 planned as-is, since they were written for PostgreSQL. It is the only **non-synthetic** workload in the project: real IMDB data with real skew and correlation, which is exactly what TPC-H's uniform generated data lacks.

Item 35 — concurrency

22 TPC-H queries at 1, 2, 4 and 8 simultaneous clients on the M1 (8 cores).

clients	total throughput (q/s)	median query slowdown
1	74.04	1.00×
2	87.06	2.02×
4	126.99	2.82×
8	153.01	4.69×

Median concurrency elasticity +0.752, range +0.286 to +0.984. Eight clients buy 2.07× throughput, so contention is real but not total — and critically, **the range is wide**: some queries scale nearly freely (0.286) while others are fully serialised (0.984). A single global contention factor will not capture it, so concurrency is a genuine per-query axis the project has never modelled.

Sanity check passed: 0 of 22 queries changed plan across concurrency levels, which is the correct answer — concurrency is not a planner input. A non-zero count would have indicated statistics drifting mid-sweep.

Item 36 — PostgreSQL version matrix

The same 22 queries on 14.19, 15.17, 17.11 and 18.6, over byte-identical data loaded from the same CSVs with the same collation, so **only the engine differs**.

version	median runtime vs PG15	queries with a different plan vs PG15
14.19	0.796×	4/22
15.17	1.000 (ref)	—
17.11	**0.708×**	2/22
18.6	0.740×	**9/22**

12 of 22 queries get a different plan on at least one version, on identical hardware and identical data. And PG15 — the version every measurement in this project was taken on — is the **slowest of the four**, with 17 and 18 roughly 30% faster.

This matters more than it looks. A ~30% runtime shift and a plan change on over half the workload is **comparable in magnitude to the hardware effects the project is trying to predict**. So "production" is a pair — hardware *and* engine version — and a two-scalar hardware signature cannot express it. Any

deployment comparing a dev box to a production server on a different PostgreSQL version is measuring both effects and attributing all of it to hardware. This is a direct parallel to Tier 3's plan-drift finding, arrived at from a different direction.

Two incidental obstacles worth recording: `pg_dump` must be at least the server's version (the PATH one was 14.19 against a 15.17 server), and a PG15 dump **cannot** be restored into PG14 at all, since downgrades are unsupported — which is why every version is loaded from source CSVs instead. PostgreSQL 17+ on macOS also refuses to start with an unset locale ("postmaster became multithreaded during startup"); `LC_ALL=C` fixes it, and setting it uniformly keeps collation — and therefore sort costs — identical across versions.

Item 37 — raw plan JSON

`collect.py` stores the full plan JSON, not just derived scalars. The original collection kept only six numbers per execution, which is why the buffer bug had to be repaired from a local re-run rather than from the data itself.

To avoid hundreds of megabytes of near-identical duplicates it stores the full plan for the first measured repetition of each cell, a **structural signature** for every repetition, and any plan whose signature differs. That makes plan drift detectable in the data rather than requiring a separate experiment — and it is what both the concurrency sanity check and the version comparison above are built on.

Items 32 and 34 — not done, and why

Item 32 (30+ machines) is blocked by AWS, not by effort or money. The account's EC2 vCPU quota is **1.0** in both ap-south-1 and us-east-1, below AWS's own default of 5.0, and no increase has ever been requested. At 1 vCPU no instance this project needs can launch — not `c5a.xlarge` (4 vCPU), not even a `.large` (2 vCPU). Spot pricing was checked and is $0.031–0.058/hr, so 30 machines for ~3 hours is about **$5**. Money was never the constraint. The original AMI `cetp-tpch-base-v1`, the `cloud` key pair and its `.pem` all still exist, so the collection can run as soon as capacity is granted.

Item 34 (SF10/SF100) was traded away: JOB (~13 GB) and SF10 TPC-H (~25 GB) do not both fit in 31 GB of free disk, and the author chose query diversity. That was the right call — Tier 1's clearest positive result was plan-structure features (17.29% → 13.29% on unseen queries), and that result is limited by query count, not by scale-factor coverage.

Item 38 — packaged, not published

`scripts/collection/package_dataset.py` produces `dist/cetp-datasets.tar.gz` plus a `MANIFEST.json` with SHA-256 for all 161 files, and DATASHEET.md follows the Gebru et al. *Datasheets for Datasets* structure, with every known defect listed rather than quietly fixed.

Deliberately **excluded**: the IMDB data (third-party, not ours to redistribute — the source URL and a checksum are recorded instead), the JOB queries, tpcds-kit, the quarantined buffer-bug dataset, and the version-coupled `.pkl` files.

Publication is left to the author — it needs their Zenodo or HuggingFace account, and one thing genuinely needs checking first: `data/external/tpcds-kit/EULA.txt`, before the generated TPC-DS SQL is redistributed.

What Tier 2 changes

The two completed measurement axes both point the same way as Tier 1 and Tier 3:

- **Concurrency sensitivity is query-specific** (+0.286 to +0.984), so it cannot be folded into a single hardware scalar.
- **Engine version changes plans on 12/22 queries and runtime by ~30%**, on fixed hardware, so it cannot be folded into a hardware scalar either.

Both are unmodelled confounds of the same order as the effect being predicted. Combined with Tier 1's finding that the two hardware axes are 96% correlated, the picture is consistent: **the "hardware signature" abstraction is carrying far less than the model assumes, and several things it does not represent at all matter just as much.**

The 234-query workload is the main asset Tier 2 delivers. It is what makes the plan-structure feature result testable at a sample size that can support it.